



Dynamic Hashing



Dynamic Hashing

❑ Hashing in DBMS

- Hashing is a DBMS technique for searching for needed data on the disc without utilizing an index structure.
- The hashing method is basically used to index items and retrieve them in a DB
- It is a method for calculating the direct position of an information record on the disk without the use of an index structure.
- To generate the actual address of a data record, hash functions containing search keys as parameters are used.



Dynamic Hashing

❑ Main Components of Hashing

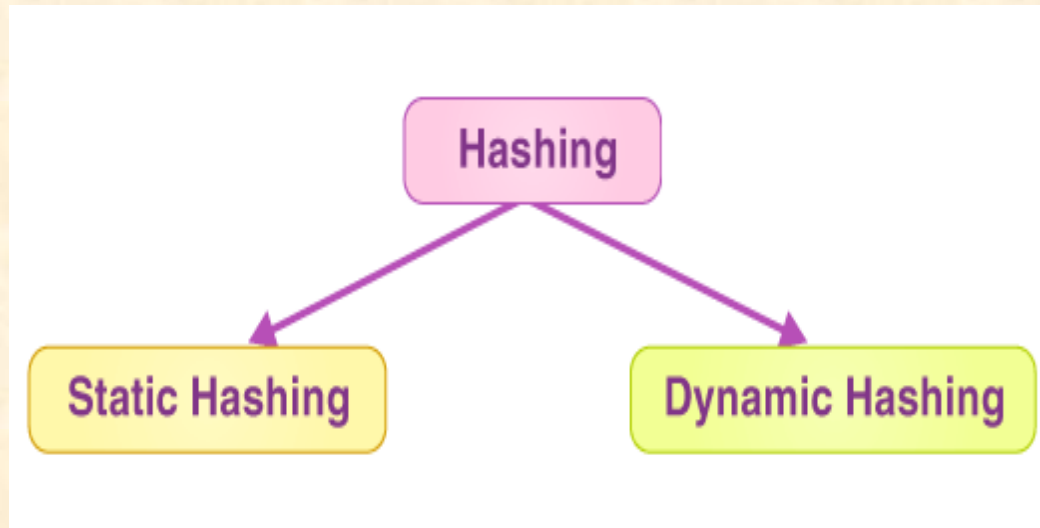
- **Bucket** – A bucket stores one entire disc block, which can store one or more records.
- **Hash Function** – A hash function, abbreviated as h , refers to a mapping function that connects all of the search-keys K to that address in which the actual records are stored.
- From the search keys to the bucket addresses, it's a function.

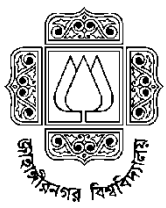


Dynamic Hashing

❑ Types of Hashing

- Hashing is of the following types:

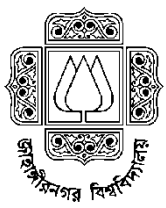




Dynamic Hashing

❑ Static Hashing in DBMS

- Whenever a search-key value is specified in static hashing, the hash algorithm always returns the same address. For example, the mod-5 hash function will generate only 5 values. For this function, the output address must always be the same.
- At all times, the number of buckets available remains constant throughout the process of static hashing
- The resultant data bucket address with static hashing will always be the same. That is, if we use the hash function mod-5 to obtain an address for EMP ID =103, we will always get the same bucket address 3. The bucket address will not change in this case.



Dynamic Hashing

❑ Static Hashing Operations

- **Search a Record:** When a record is needed, the very same hash function is used in order to get the address of that bucket in which the data is kept.
- **Insert a Record:** When a new record is entered into the table, the hash key is used to construct an address for the new record, and the record is placed there.
- **Delete a Record:** To delete a record, we must first retrieve the record that will be destroyed. The records for this address will then be deleted from memory.
- **Update a Record:** To edit a record, we'll use a hash function to find it first, then change the data record.



Dynamic Hashing

❑ Static Hashing Operations

- If we wish to add a new record to the file, but the address of the data bucket formed by the hash function isn't empty, or information already exists in that address, we can't add the record. Bucket overflow is a term used in static hashing to describe this occurrence. This is a critical condition in this strategy.
- To deal with this scenario, the most widely utilized methods are :

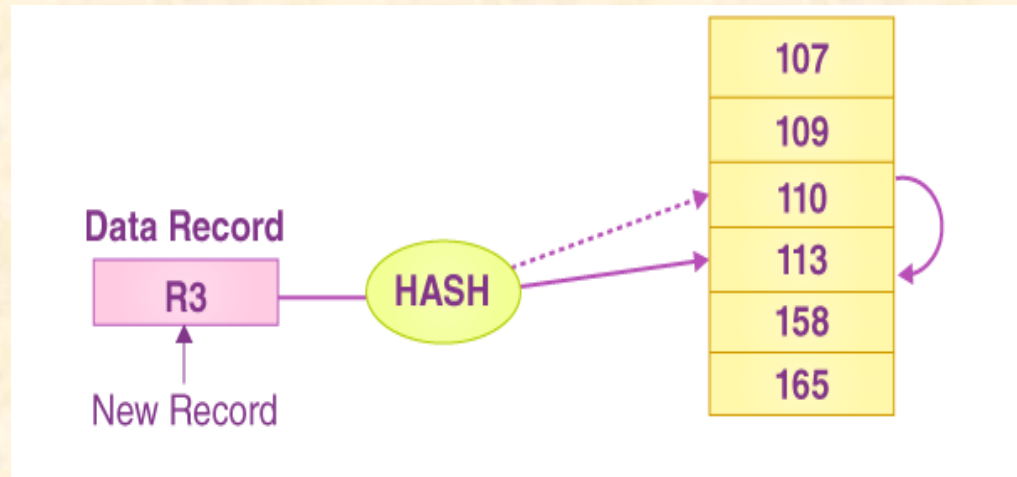
Open hashing and
Closed hashing



Dynamic Hashing

❑ Open Hashing

- Whenever a hash function generates any address that already contains data, the next bucket is assigned to it, and this process is called **Linear probing**.
- For instance, if the record R3 needs to be entered, the hash function will generate 110 as R3's address. However, the address is already full; as a result, the system selects 113 as the next available data bucket and assigns R3 to it.

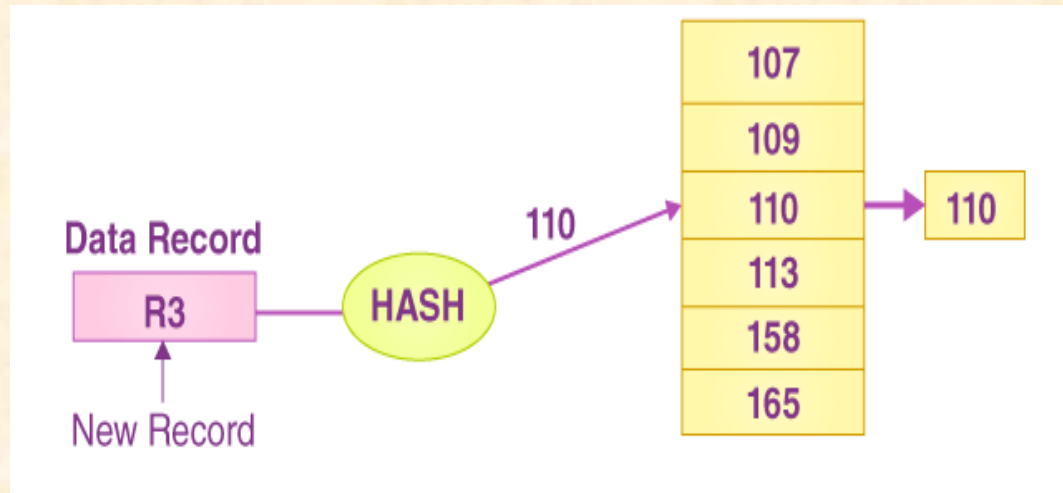




Dynamic Hashing

❑ Closed Hashing

- When a data bucket is full, a new one is created and connected after the old one, and this method is called ***Overflow chaining***.
- For example, if the record R3 needs to be added to the database, the hash function will assign it with the address 110. However, this bucket is too full to accommodate the additional data. In this scenario, a new bucket is placed and linked to the end of 110 buckets.





Dynamic Hashing

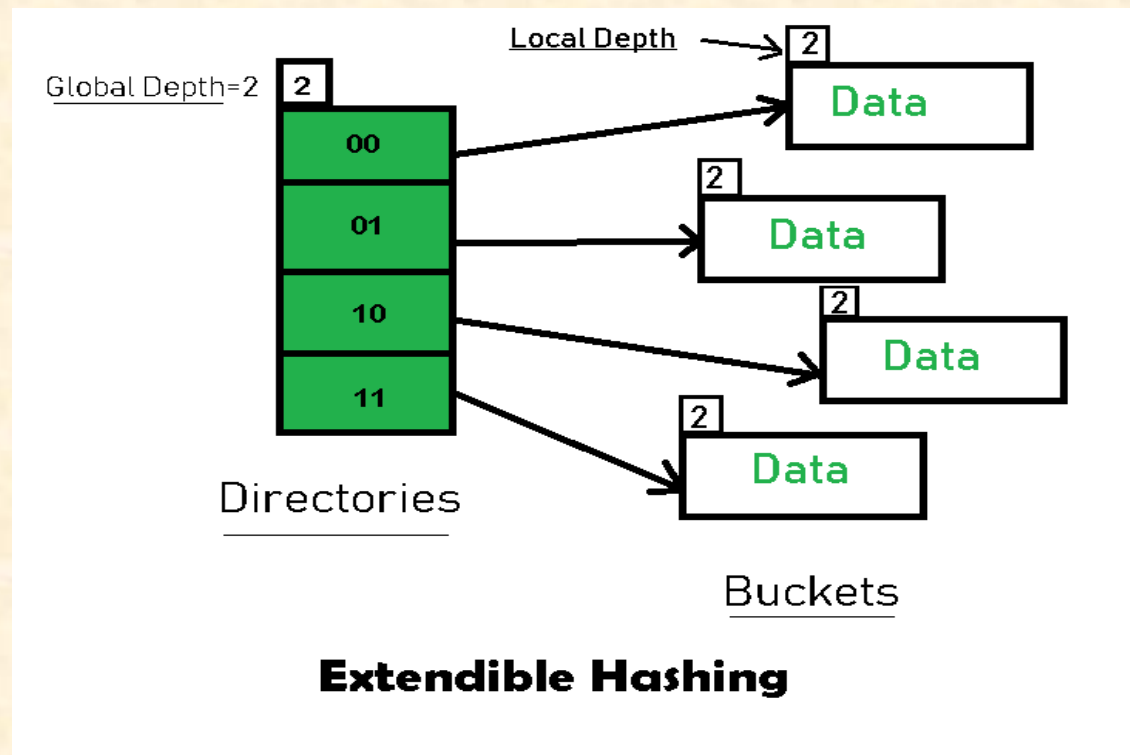
❑ Dynamic Hashing in DBMS

- The disadvantage of static hashing is that it doesn't expand or contract dynamically as the database size grows or diminishes.
- Dynamic hashing is a technology that allows data buckets to be created and withdrawn on the fly. It is also known as *Extendible hashing*.
- In dynamic hashing, the hash function is designed to output a huge number of values, but only a few are used at first.
- Dynamic hashing is a mechanism for dynamically adding and removing data buckets on demand.



Dynamic Hashing

❑ Basic Structure of Extendible Hashing





Dynamic Hashing

❑ Basic Structure of Extendible Hashing

- **Directories:** These containers store pointers to buckets. Each directory is given a unique *id* which may change each time when expansion takes place. The hash function returns this directory *id* which is used to navigate to the appropriate bucket. Number of Directories = $2^{\text{Global Depth}}$.
- **Buckets:** They store the hashed keys. Directories point to buckets. A bucket may contain more than one pointers to it if its local depth is less than the global depth.



Dynamic Hashing

❑ Basic Structure of Extendible Hashing

- **Global Depth:** It is associated with the Directories. They denote the number of bits which are used by the hash function to categorize the keys. Global Depth = Number of bits in directory *id*.
- **Local Depth:** It is the same as that of Global Depth except for the fact that Local Depth is associated with the buckets and not the directories. Local depth in accordance with the global depth is used to decide the action that to be performed in case an overflow occurs. Local Depth is always less than or equal to the Global Depth.



Dynamic Hashing

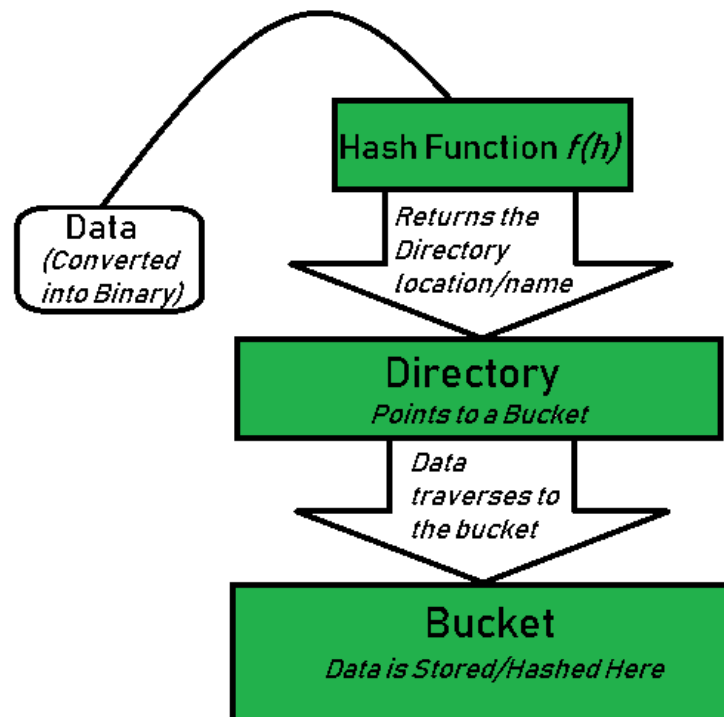
❑ Basic Structure of Extendible Hashing

- **Bucket Splitting:** When the number of elements in a bucket exceeds a particular size, then the bucket is split into two parts.
- **Directory Expansion:** Directory Expansion Takes place when a bucket overflows. Directory Expansion is performed when the local depth of the overflowing bucket is equal to the global depth.



Dynamic Hashing

❑ Basic Working of Extendible Hashing





Dynamic Hashing

❑ Basic Working of Extendible Hashing

– Step 1 – Analyze Data Elements:

Data elements may exist in various forms eg. Integer, String, Float, etc. Currently, let us consider data elements of type integer. eg: 49.

– Step 2 – Convert into binary format:

Convert the data element in Binary form. For string elements, consider the ASCII equivalent integer of the starting character and then convert the integer into binary form. Since we have 49 as our data element, its binary form is 110001.



Dynamic Hashing

❑ Basic Working of Extendible Hashing

– Step 3 – Check Global Depth of the directory.

Suppose the global depth of the Hash-directory is 3.

– Step 4 – Identify the Directory:

Consider the ‘Global-Depth’ number of LSBs in the binary number and match it to the directory *id*. For Example, the binary value of 49 is 110001 and the global-depth is 3. So, the hash function will return 3 LSBs of 110**001** that is, 001.

– Step 5 – Navigation: Now, navigate to the bucket pointed by the directory with directory-*id* 001.



Dynamic Hashing

❑ Basic Working of Extendible Hashing

– Step 6 – Insertion and Overflow Check:

Insert the element and check if the bucket overflows. If an overflow is encountered, go to **step 7** followed by **Step 8**, otherwise, go to **step 9**.

– Step 7 – Tackling Overflow Condition during Data Insertion:

Many times, while inserting data in the buckets, it might happen that the Bucket overflows. In such cases, we need to follow an appropriate procedure to avoid mishandling of data.



Dynamic Hashing

❑ Basic Working of Extendible Hashing

First, Check if the local depth is less than or equal to the global depth. Then choose one of the cases below.

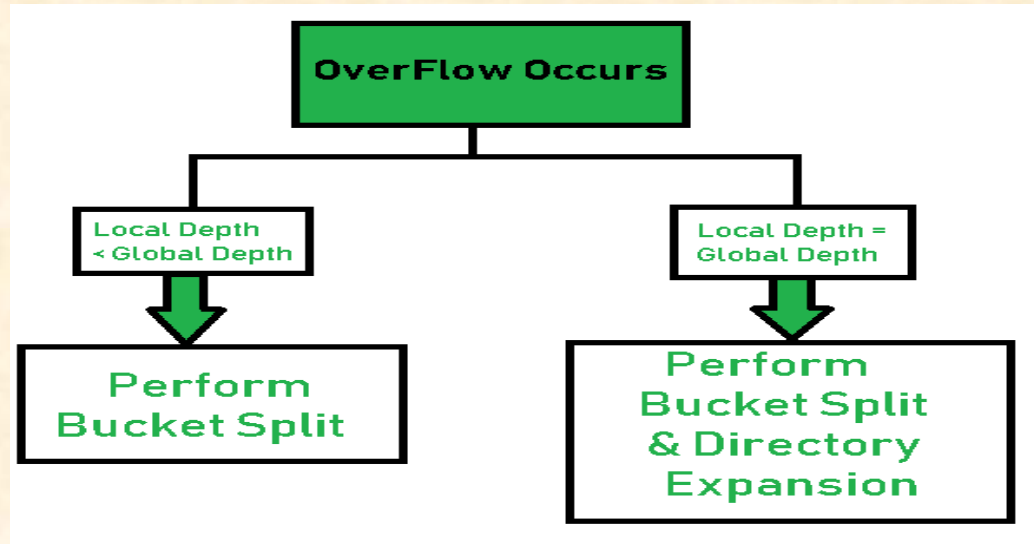
Case-1: If the local depth of the overflowing Bucket is equal to the global depth, then Directory Expansion, as well as Bucket Split, needs to be performed. Then increment the global depth and the local depth value by 1. And, assign appropriate pointers. Directory expansion will double the number of directories present in the hash structure.

Case-2: In case the local depth is less than the global depth, then only Bucket Split takes place. Then increment only the local depth value by 1. And, assign appropriate pointers.



Dynamic Hashing

❑ Basic Working of Extendible Hashing



- **Step 8 – Rehashing of Split Bucket Elements:** The Elements present in the overflowing bucket that is split are rehashed w.r.t the new global depth of the directory.
- **Step 9** – The element is successfully hashed.



Dynamic Hashing

❑ Algorithm for Inserting New Record into the Dynamic/Extendible Hash Structure

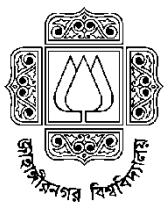
– The following steps explain how the algorithm works:

Step-1: Initialize the bucket depths and the global depth of the directories.

Step-2: Convert data into a binary representation.

Step-3: Consider the "global depth" number of the least significant bits (LSBs) of data.

Step-4: Map the data according to the ID of a directory.



Dynamic Hashing

❑ Algorithm for Inserting New Record into the Dynamic/Extendible Hash Structure

Step-5: Check for the following conditions if a bucket overflows (if the number of elements in a bucket exceeds the set limit):

- i. **If Global depth = bucket depth:** Split the bucket into two and increment the global depth and the buckets' depth. Re-hash the elements that were present in the split bucket.
- ii. **Global depth > bucket depth:** Split the bucket into two and increment the bucket depth only. Re-hash the elements that were present in the split bucket.

Step-6: Repeat the steps above for each element.



Dynamic Hashing

❑ Searching Algorithm

When searching for a key, the hash function computes the key's hash value, and the resulting index is used to quickly locate the key's value in the table.

- **Input:** Key k and hash table.
- **Compute Hash:** Compute the hash value h for the key.

$$h = \text{hash_function}(k)$$

- **Determine Index:** Use the first n bits of h to find the directory index.

$$\text{index} = h[0:n]$$

- **Locate Bucket:** Access the bucket at the computed index.
- **Search in Bucket:**
 - Check if the key k exists in the bucket.
 - If found, return the corresponding value.
 - If not found, return an error message (key does not exist).



Dynamic Hashing

❑ Example

Consider the following elements:

16, 4, 6, 22, 24, 10, 31, 7, 9, 20, 26

Construct the extendible hash structure for Bucket Size=3.

Solution:



Dynamic Hashing

❑ Example

Solution: Calculate the binary forms of each of the given key elements.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010

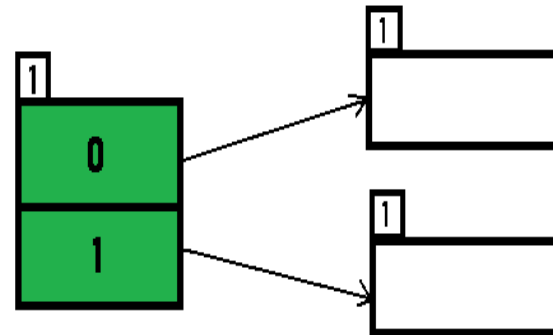


Dynamic Hashing

❑ Example

Solution: Initially, the global-depth and local-depth is always 1. Thus, the hashing frame looks like this:

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



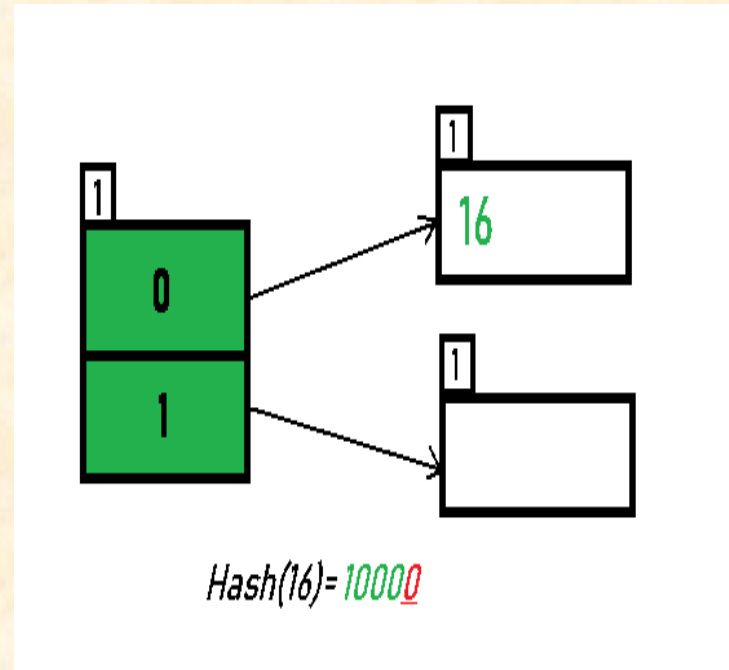


Dynamic Hashing

❑ Example

Inserting 16: The binary format of 16 is 10000 and global-depth is 1. The hash function returns 1 LSB of 10000 which is 0. Hence, 16 is mapped to the directory with id=0.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



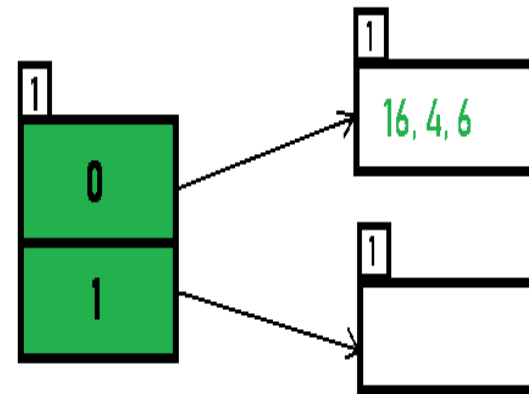


Dynamic Hashing

❑ Example

Inserting 4 and 6: Both 4(00100) and 6(00110) have 0 in their LSB. Hence, they are hashed as follows:

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



$Hash(4)=100$

$Hash(6)=110$

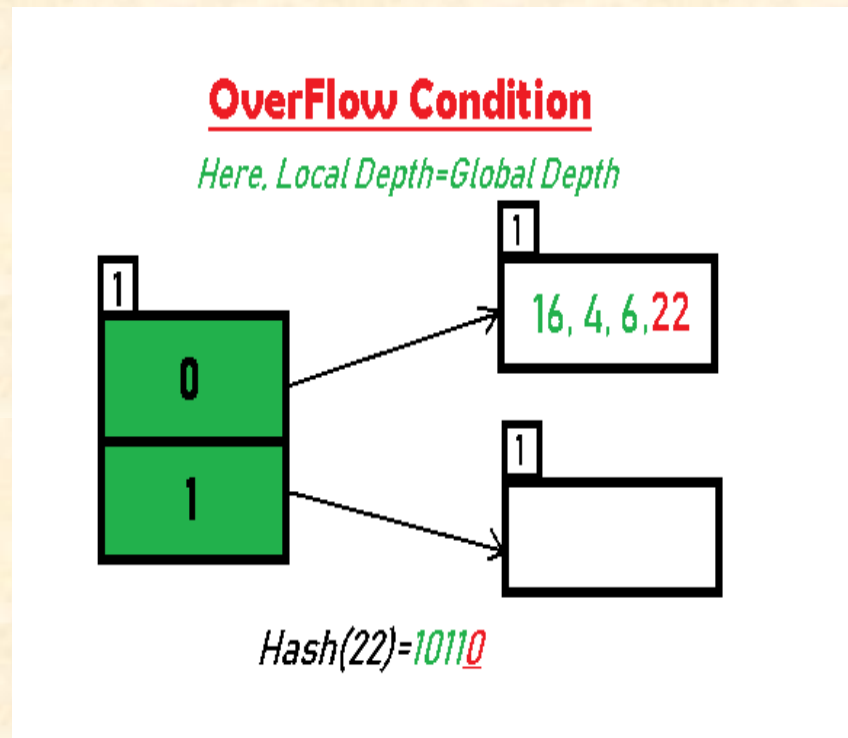


Dynamic Hashing

❑ Example

Inserting 22: The binary form of 22 is 10110. Its LSB is 0. The bucket pointed by directory 0 is already full. Hence, **Overflow occurs**.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



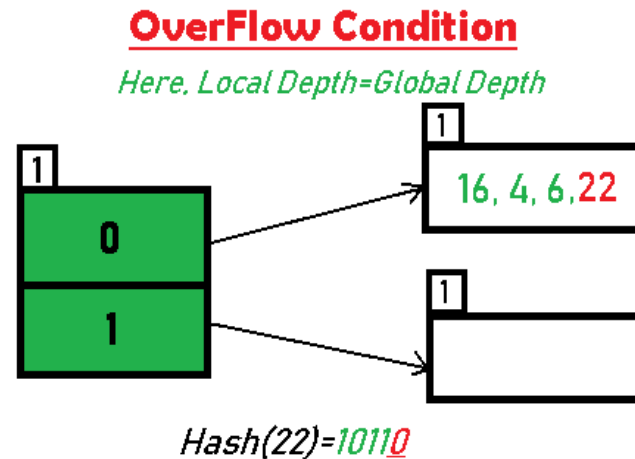


Dynamic Hashing

❑ Example

As directed by **Step 7-Case 1**, Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Also, rehashing of numbers present in the overflowing bucket takes place after the split. And, since the global depth is incremented by 1, now, the global depth is 2. Hence, 16, 4, 6, 22 are now rehashed w. r. t 2 LSBs. [16(100**00**), 4(001**00**), 6(001**10**), 22(101**10**)].

Key element	Binary form
16	100 00
4	001 00
6	001 10
22	101 10
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010





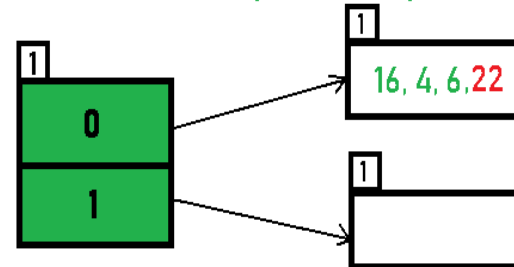
Dynamic Hashing

□ Example

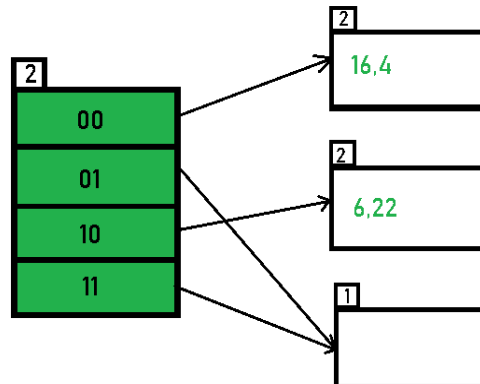
Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010

OverFlow Condition

Here, Local Depth=Global Depth



After Bucket Split and Directory Expansion

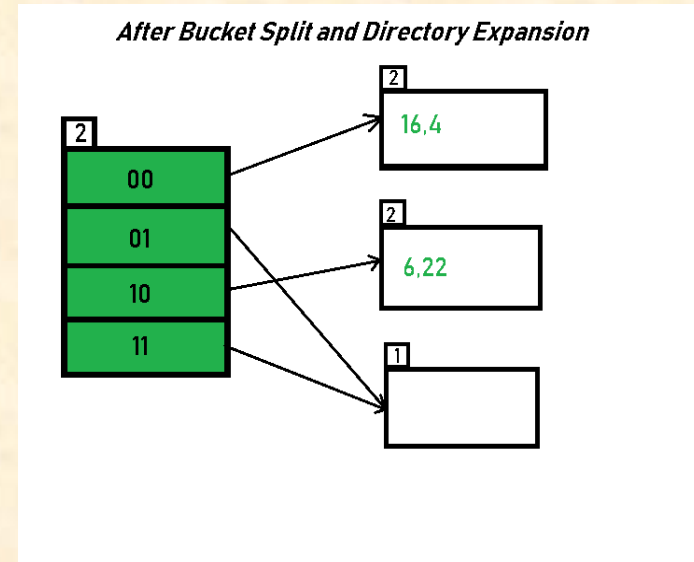




Dynamic Hashing

❑ Example

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



The bucket which was underflow has remained untouched. The number of directories has doubled, we now have 2 directories 01 and 11 pointing to the same bucket. This is because the local-depth of the bucket has remained 1. Any bucket having a local depth less than the global depth is pointed-to by more than one directories.

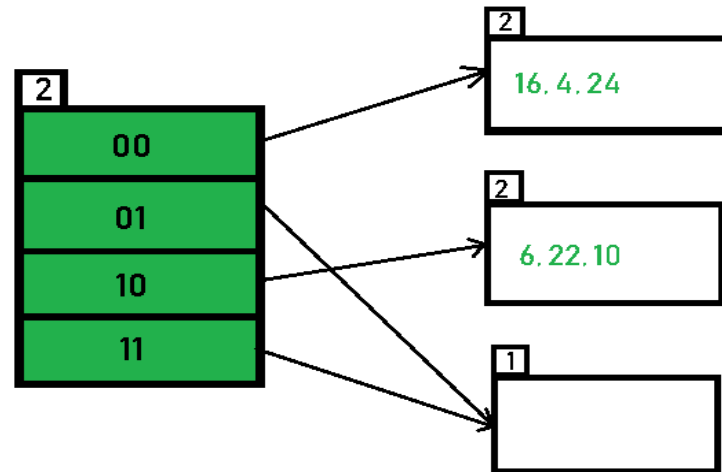


Dynamic Hashing

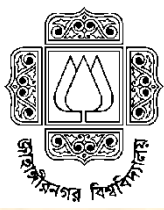
❑ Example

Inserting 24 and 10: 24(110**00**) and 10 (010**10**) can be hashed based on directories with id 00 and 10. Here, we encounter no overflow condition.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	110 00
10	010 10
31	11111
7	00111
9	01001
20	10100
26	11010



$Hash(24) = 110**00**$
 $Hash(10) = 010**10**$

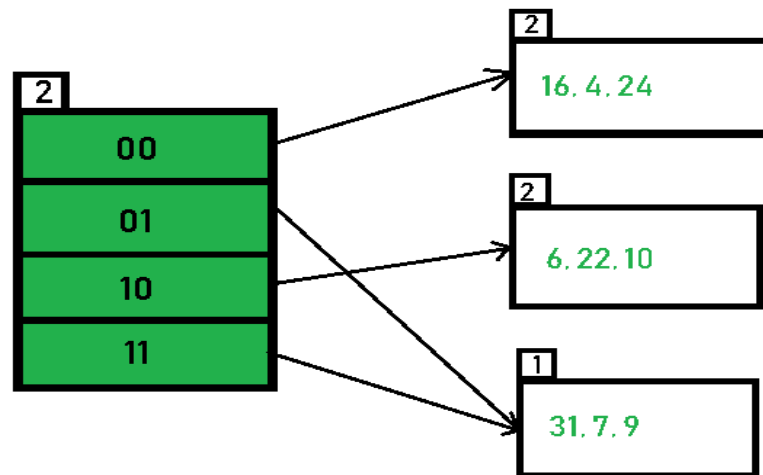


Dynamic Hashing

❑ Example

Inserting 31, 7, 9: All of these elements [31(111**11**), 7(001**11**), 9(010**01**)] have either 01 or 11 in their LSBs. Hence, they are mapped on the bucket pointed out by 01 and 11. We do not encounter any overflow condition here.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



$Hash(31) = 11111$
 $Hash(7) = 111$
 $Hash(9) = 1001$

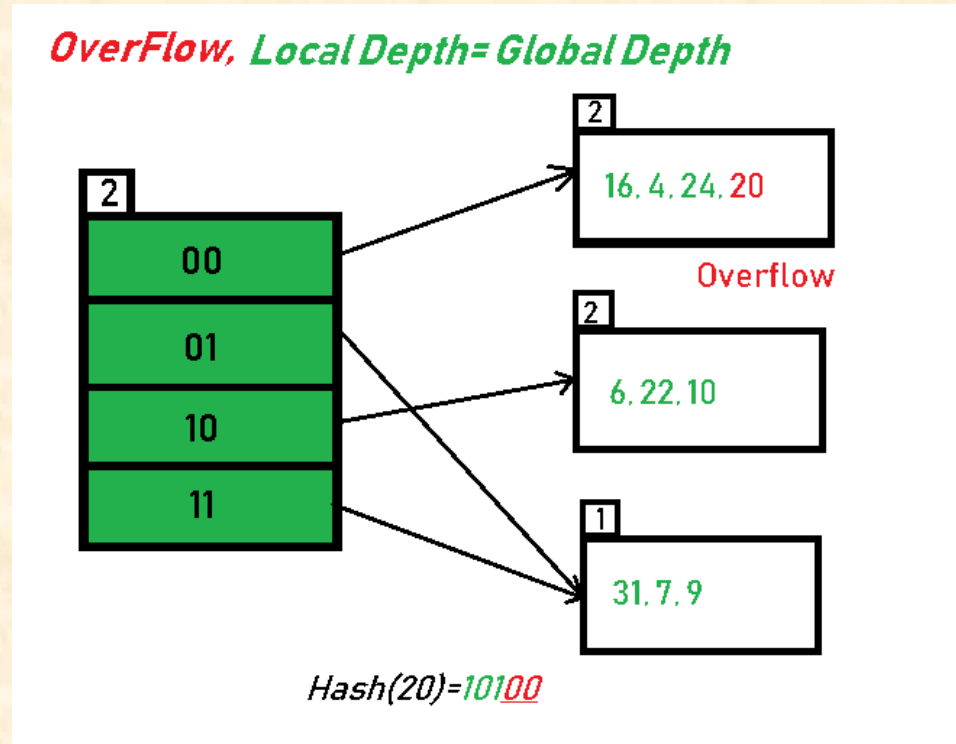


Dynamic Hashing

❑ Example

Inserting 20: Insertion of data element 20 (101**00**) will again cause the overflow problem.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



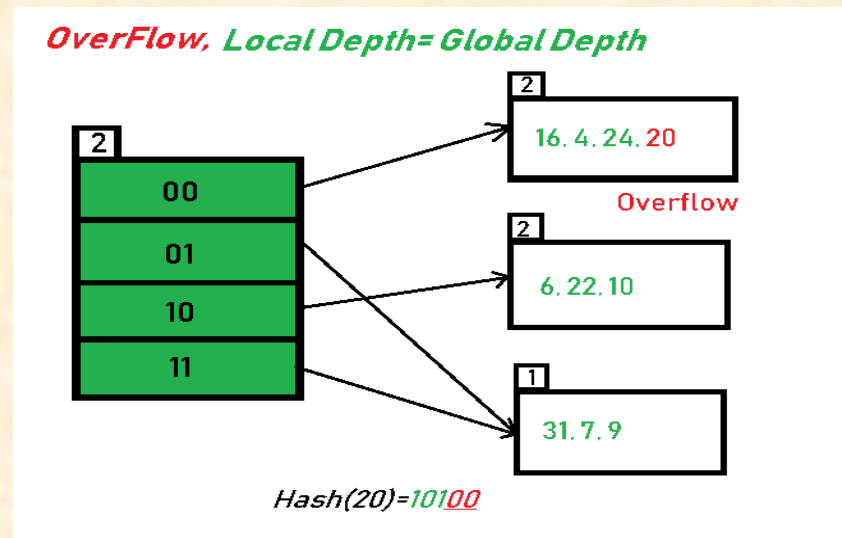


Dynamic Hashing

❑ Example

20 is inserted in bucket pointed out by 00. As directed by **Step 7-Case 1**, since the **local depth of the bucket = global-depth**, directory expansion (doubling) takes place along with bucket splitting. Elements present in overflowing bucket are rehashed with the new global depth.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010





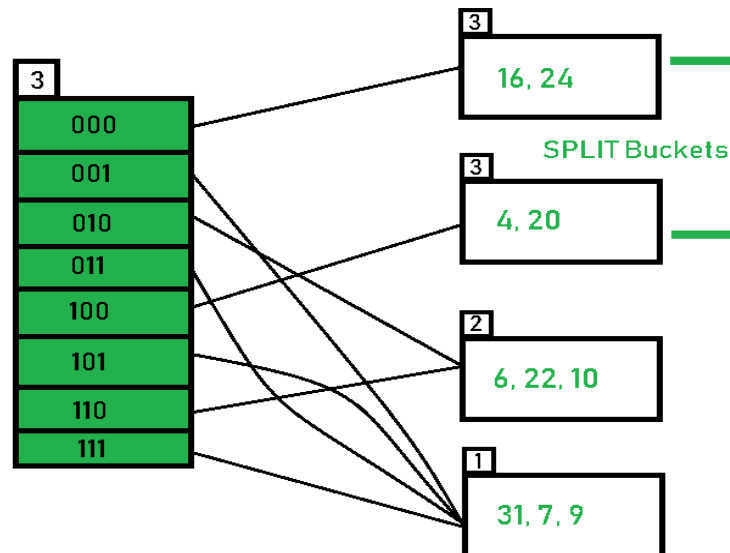
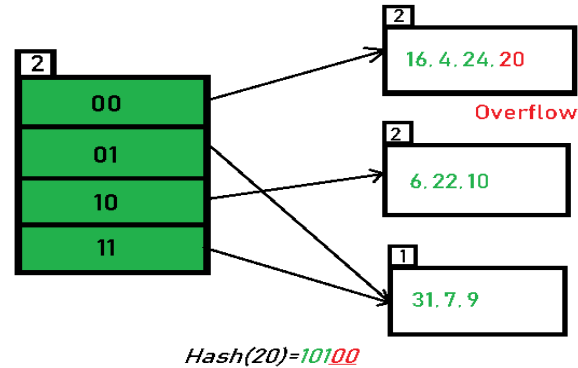
Dynamic Hashing

❑ Example

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010

Now, the new Hash table looks like this.

Overflow, Local Depth = Global Depth





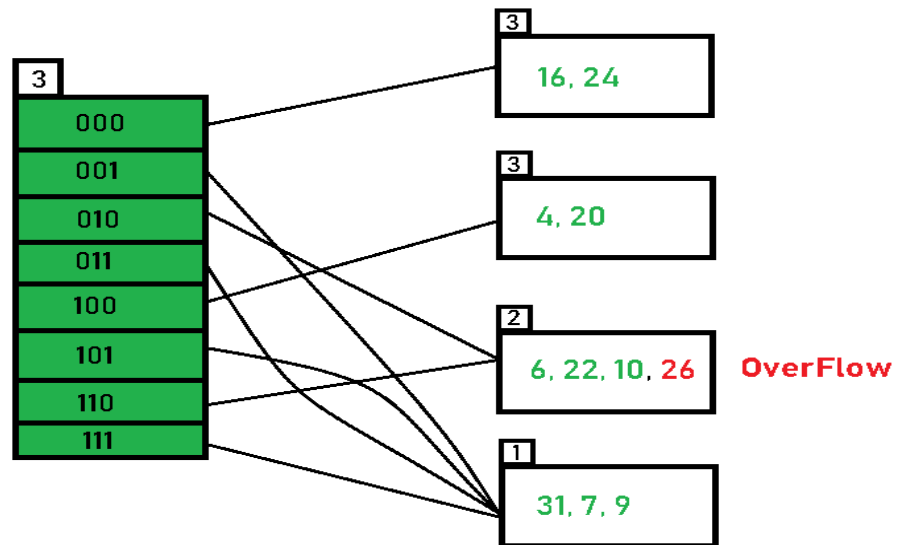
Dynamic Hashing

❑ Example

Inserting 26: Global depth is 3. Hence, 3 LSBs of 26(11**010**) are considered. Therefore 26 best fits in the bucket pointed out by directory 010. **Overflow occurs.**

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11 010

Hash(26)=11**010**
Overflow, Local Depth < Global Depth



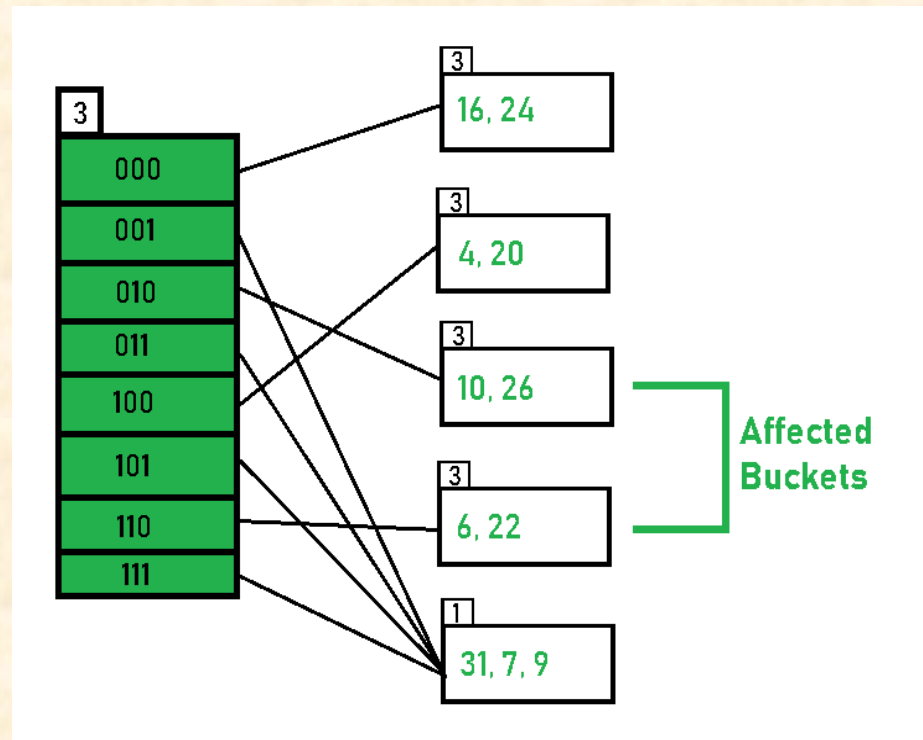


Dynamic Hashing

❑ Example

As directed by **Step 7-Case 2**, since the **local depth of bucket** < **Global depth** ($2 < 3$), directories are not doubled but, only the bucket is split and elements are rehashed.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010



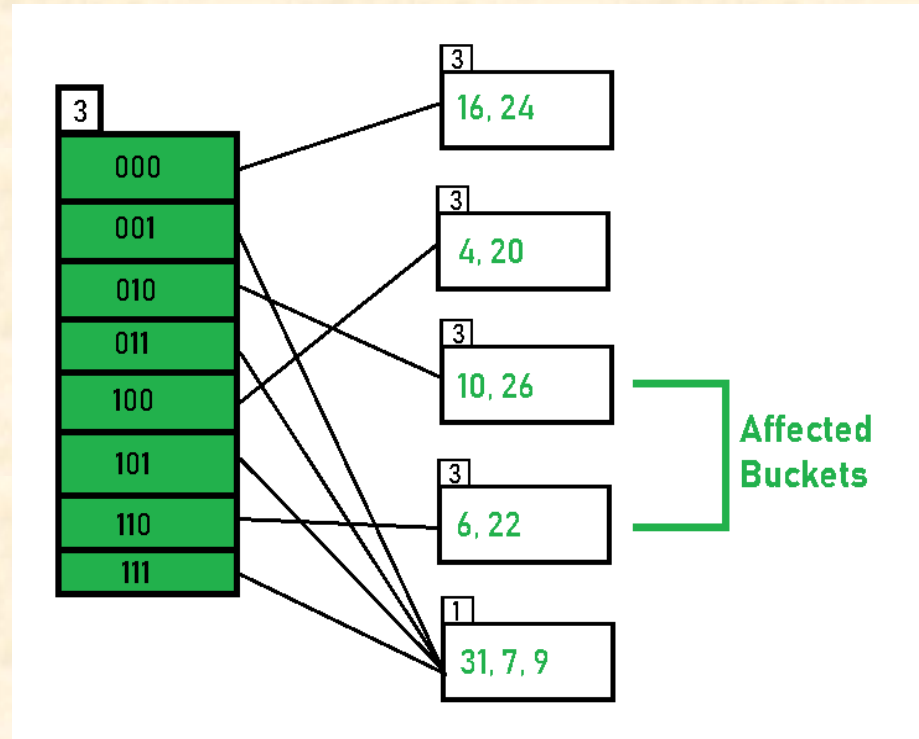


Dynamic Hashing

❑ Example

Finally, the output of hashing the given list of numbers is obtained.

Key element	Binary form
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	11010





Dynamic Hashing

□ Key Observations

- A Bucket will have more than one pointers pointing to it if its local depth is less than the global depth.
- When overflow condition occurs in a bucket, all the entries in the bucket are rehashed with a new local depth.
- The size of a bucket cannot be changed after the data insertion process begins.



Dynamic Hashing

❑ Pros and Cons of Dynamic Hashing

Advantages: **Dynamic Hashing Pros**

- Data retrieval is less expensive (in terms of computing).
- No problem of Data-loss since the storage capacity increases dynamically.
- With dynamic changes in hashing function, associated old values are rehashed w.r.t the new hash function.
- Performance will not get affected as the amount of data grows in the system. To accommodate the data, size of memory will be increased.
- Dynamic hashing improves the utilization of the memory.
- This method is efficient to handle the dynamic database where size of data changes frequently.



Dynamic Hashing

❑ Pros and Cons of Dynamic Hashing

Limitations: **Dynamic Hashing Cons**

- The directory size may increase significantly if several records are hashed on the same directory while keeping the record distribution non-uniform.
- Size of every bucket is fixed.
- Memory is wasted in pointers when the global depth and local depth difference becomes drastic.
- In dynamic hashing, bucket overflow can happen.
- This method is complicated to code.



Dynamic Hashing

❑ Indexing VS Hashing in DBMS

Data Retrieval Speed:

- **Indexing:** With its pre-organized data structures, Indexing offers faster data retrieval, especially for range queries and ordered records.
- **Hashing:** Hashing outperforms Indexing when searching for specific items, especially in large databases.

Storage Efficiency:

- **Indexing:** While Indexes provide optimized search, they come with the cost of additional storage space.
- **Hashing:** Hashing uses a dynamic allocation of buckets, ensuring better storage efficiency, particularly for databases with varying data sizes.



Dynamic Hashing

❑ Indexing VS Hashing in DBMS

Database Size:

- **Indexing:** Indexing works well for small to medium-sized databases where the additional storage overhead is manageable.
- **Hashing:** Hashing shines in large databases, where it scales efficiently and maintains performance even with massive datasets.

Complexity:

- **Indexing:** The complexity of Indexing increases with the number of indexes and their size, potentially affecting performance.
- **Hashing:** Hashing offers a simpler and more straightforward method for data retrieval, reducing complexity for certain scenarios.



Dynamic Hashing

❑ Indexing VS Hashing in DBMS

Applications: Indexing

- **Online Transaction Processing (OLTP) Systems:** Indexing is well-suited for OLTP systems, where rapid data retrieval is essential for handling numerous concurrent user requests.
- **Range Queries:** When dealing with queries involving a range of values, ordered indexing can significantly speed up search times.

Applications: Hashing

- **Large Databases:** Hashing excels in large databases, where its direct location calculation ensures fast access to records without the need for complex index structures.
- **Searching Unsorted Data:** When data is unsorted, Hashing can still efficiently retrieve desired items, whereas Indexing may require additional sorting operations.



Dynamic Hashing

❑ Example

Consider the following data to construct the dynamic/extendible hash structure:

Data = {28, 4, 19, 1, 22, 16, 12, 0, 5, 7}

Bucket limit = 3

Solution:



Dynamic Hashing

❑ Example

Solution: Convert the data into binary representation:

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111

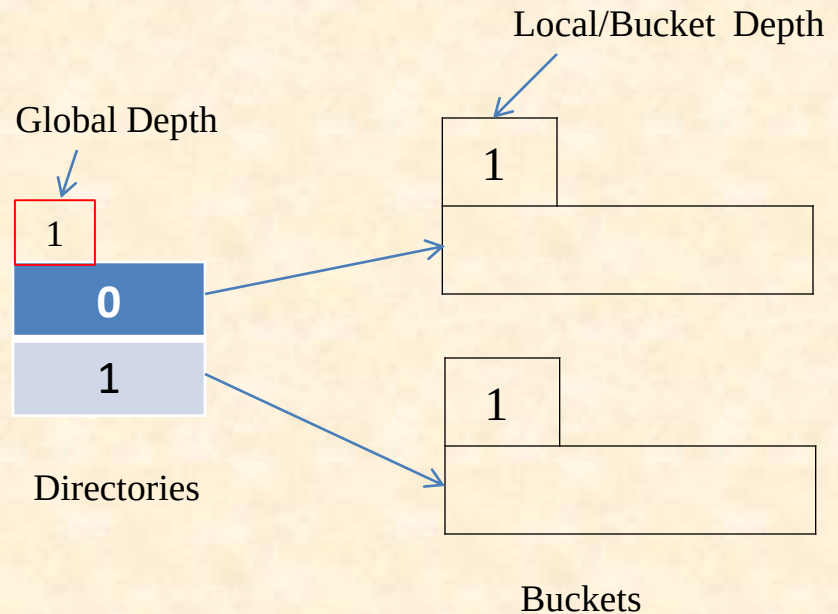


Dynamic Hashing

❑ Example

Solution: Initialize the hash table with two initial directories and buckets. Set the global depth and bucket depth to 1

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



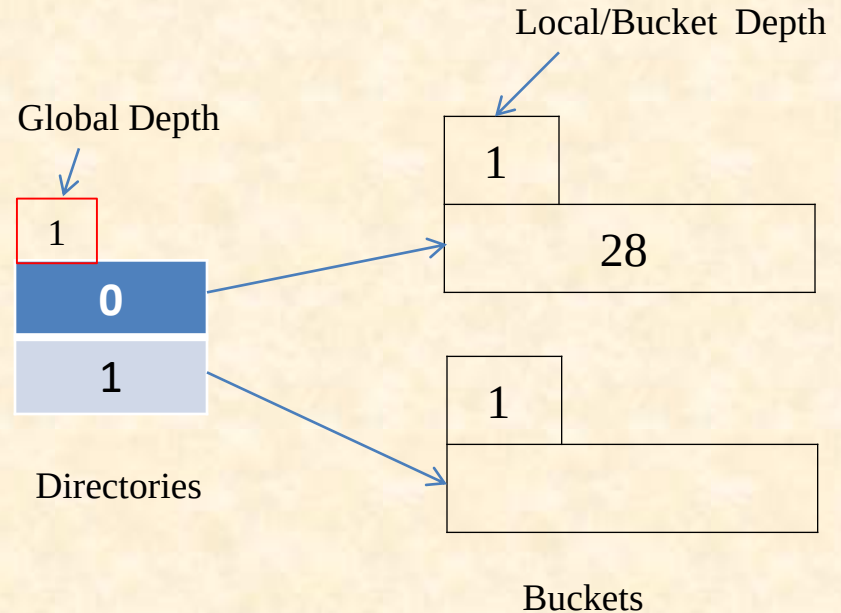


Dynamic Hashing

❑ Example

Solution: Inserting 28 (1110**0**). Global-depth is 1. The hash function returns 1 LSB of 1110**0** which is 0. Hence, 28 is mapped to the directory with id=0.

Data	Binary Form
28	1110 0
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



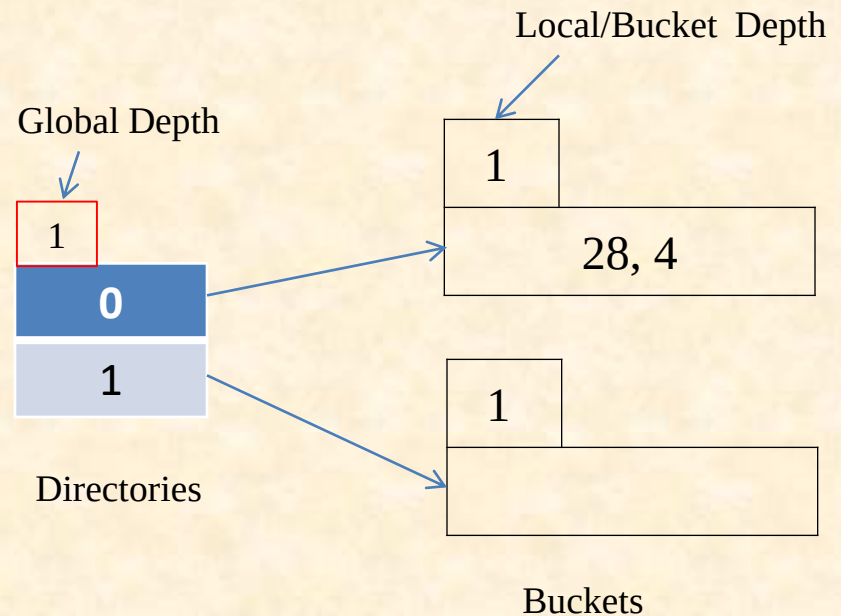


Dynamic Hashing

❑ Example

Solution: Inserting 4 (0010**0**). The hash function returns 1 LSB of 0010**0** which is 0. Hence, 4 is mapped to the directory with id=0.

Data	Binary Form
28	11100
4	0010 0
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



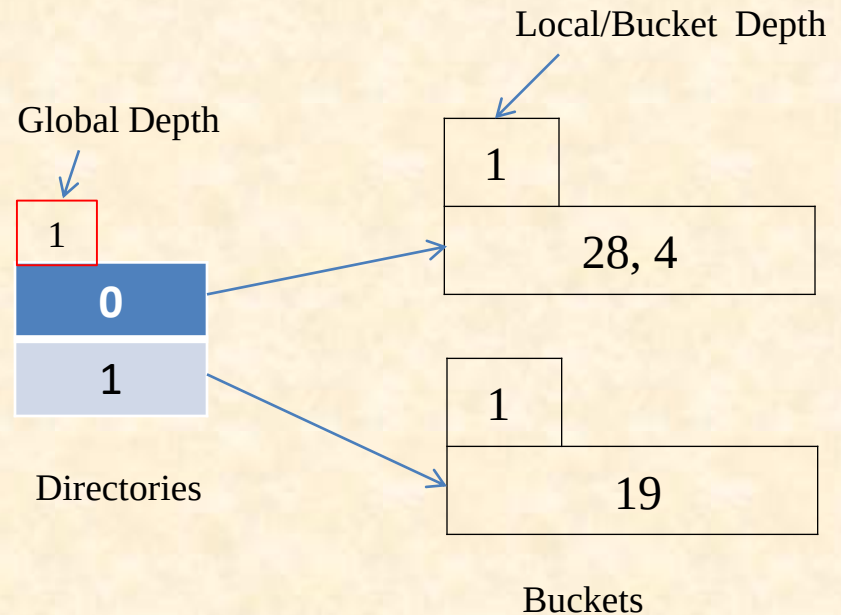


Dynamic Hashing

❑ Example

Solution: Inserting 19 (1001**1**). Add it to the "1" ID bucket as the hash function returns 1 LSB of 1001**1** which is 1.

Data	Binary Form
28	11100
4	00100
19	1001 1
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



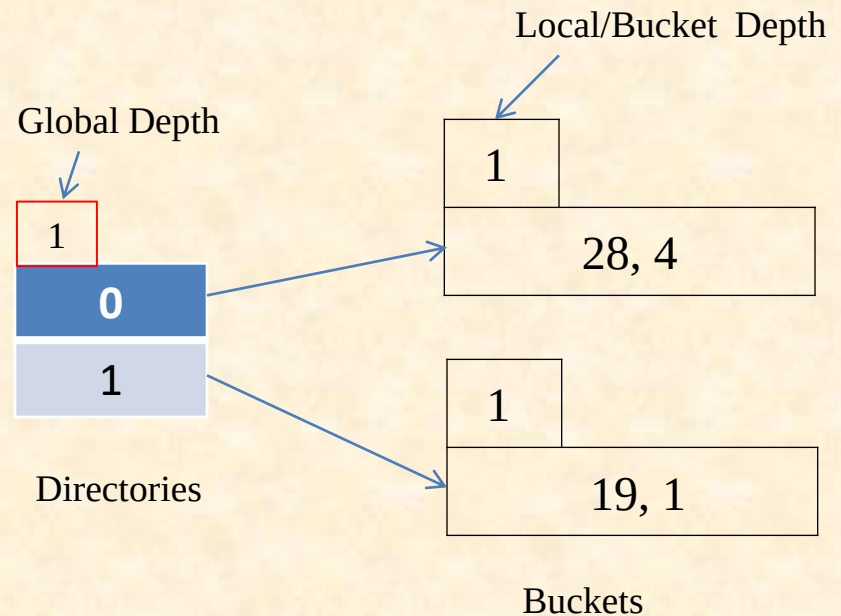


Dynamic Hashing

❑ Example

Solution: Inserting 1 (0000**1**). Add it to the "1" ID bucket as the hash function returns 1 LSB of 0000**1** which is 1.

Data	Binary Form
28	11100
4	00100
19	10011
1	0000 1
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



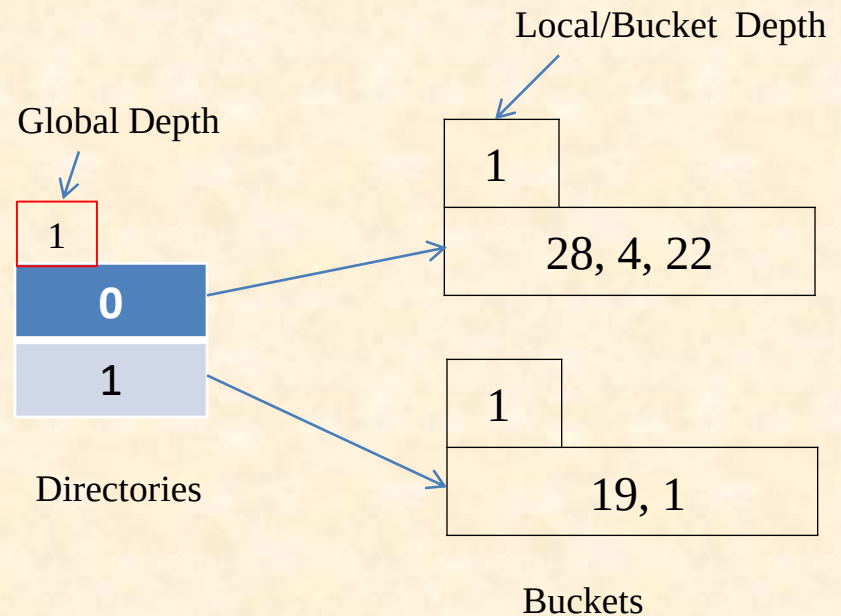


Dynamic Hashing

❑ Example

Solution: Inserting 22 (1011**0**). Add it to the "0" ID bucket as the hash function returns 1 LSB of 1011**0** which is 0.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



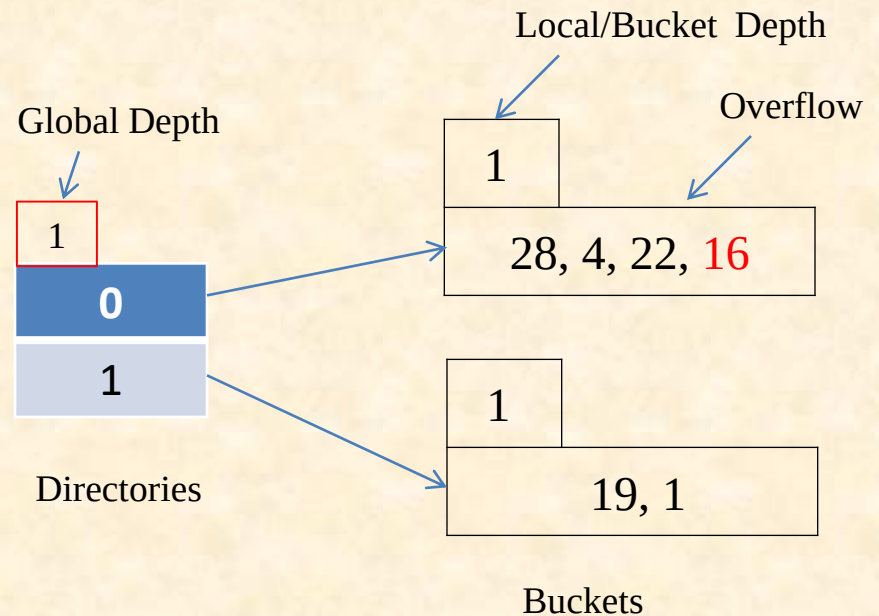


Dynamic Hashing

❑ Example

Solution: Inserting 16 (1000**0**). Adding it to the "0" ID bucket makes it overflow.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	1000 0
12	01100
0	00000
5	00101
7	00111



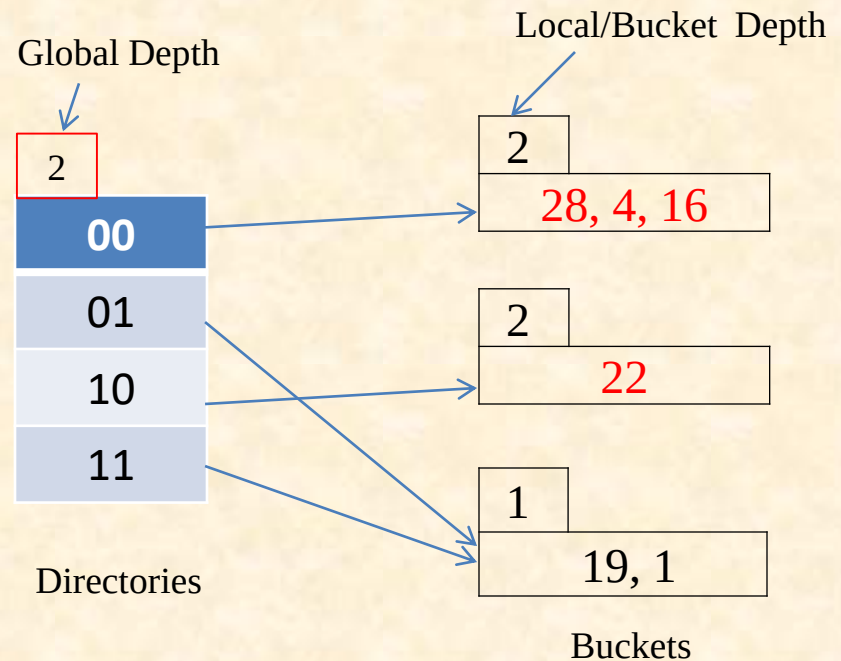


Dynamic Hashing

❑ Example

Solution: Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Rehash the numbers present in the overflowing bucket after splitting the bucket. Global depth is incremented by 1. Now, the global depth is 2. Hence, 28, 4, 22, and 16 are now rehashed w. r. t 2 LSBs.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



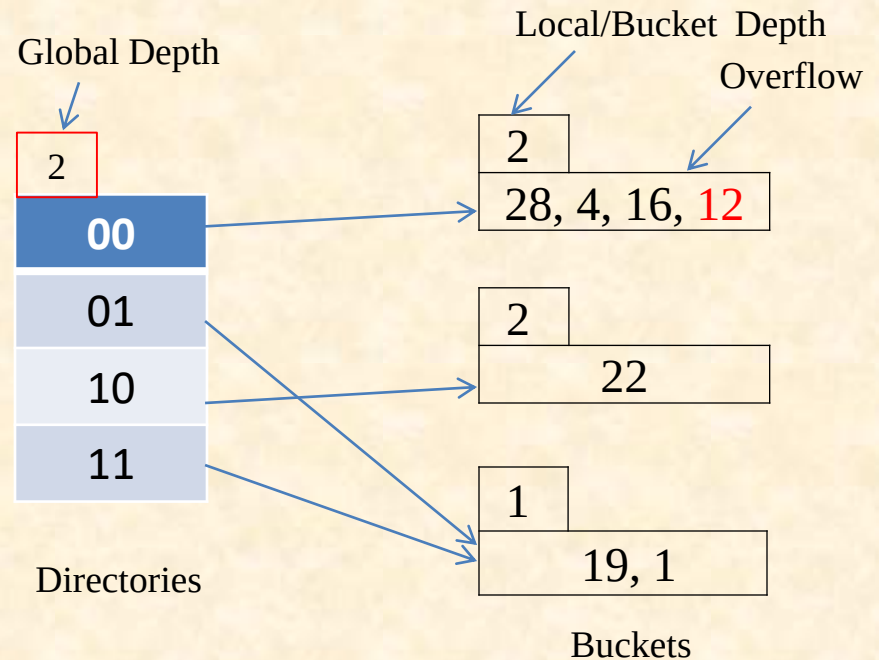


Dynamic Hashing

❑ Example

Solution: Inserting 12 (011**00**). Adding it to the "00" ID bucket makes it overflow.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	011 00
0	00000
5	00101
7	00111



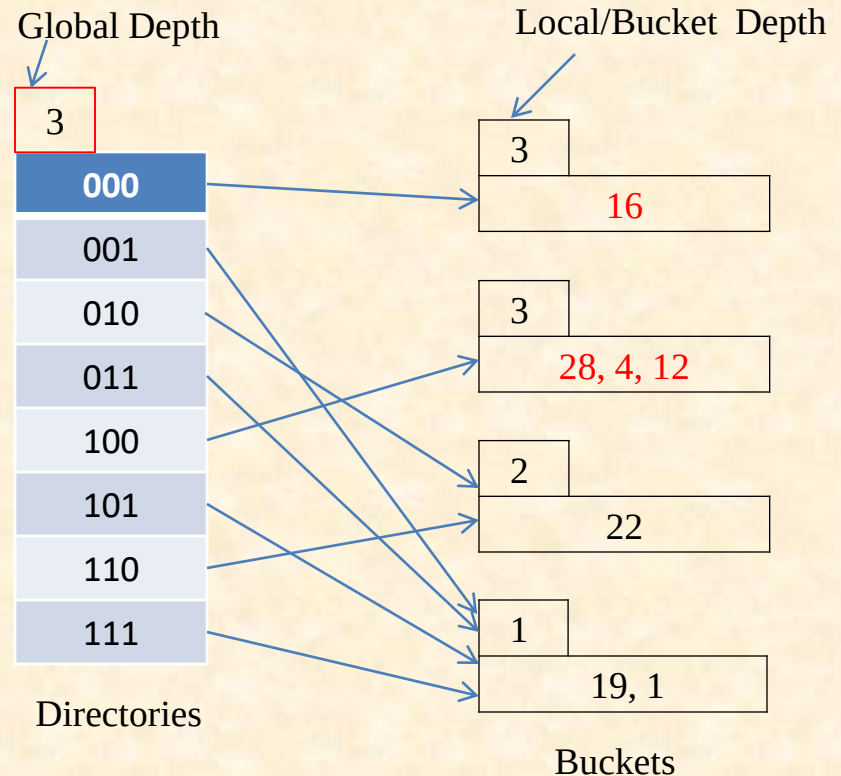


Dynamic Hashing

❑ Example

Solution: Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Rehash the numbers present in the overflowing bucket after splitting the bucket. Global depth is incremented by 1. Now, the global depth is 3. Hence, 28, 4, 16 and 12 are now rehashed w. r. t 3 LSBs.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111



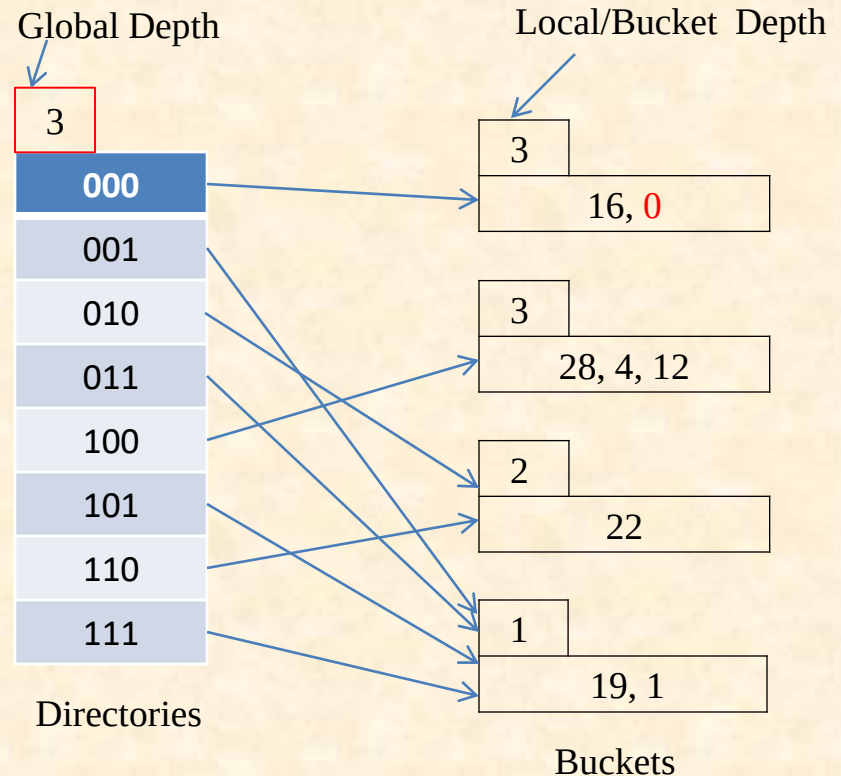


Dynamic Hashing

❑ Example

Solution: Inserting 0 (00**000**). Add it to the "000" ID bucket as the hash function returns 3 LSB of 00**000** which is 000.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00 000
5	00101
7	00111



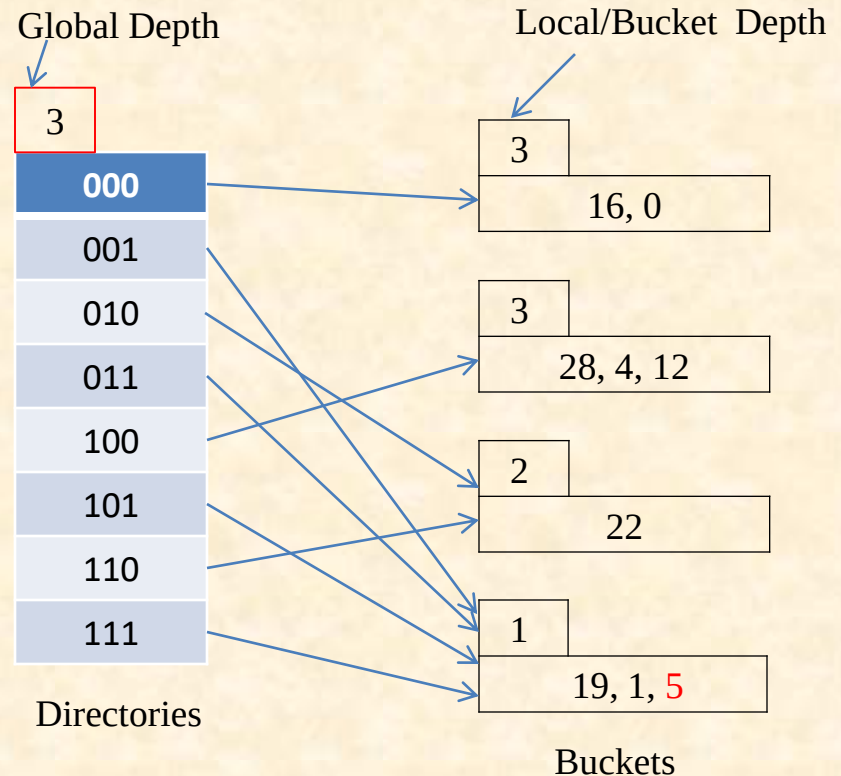


Dynamic Hashing

❑ Example

Solution: Inserting 5 (00**101**). Add it to the “101” ID bucket as the hash function returns 3 LSB of 00**101** which is 101.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00 101
7	00111



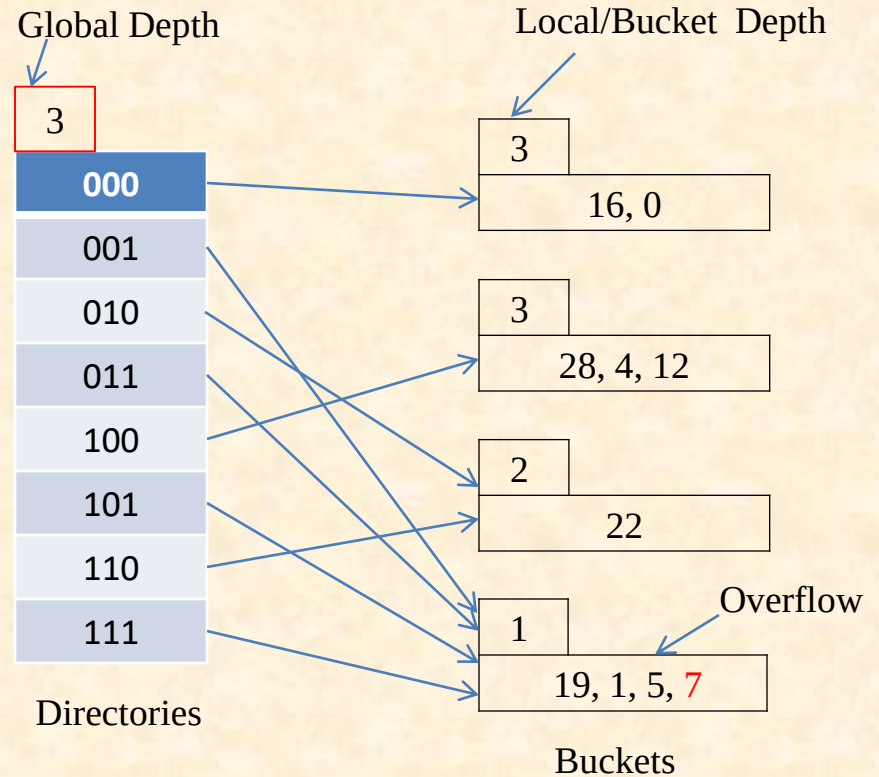


Dynamic Hashing

❑ Example

Solution: Inserting 7 (00**111**). Adding it to the “111” ID bucket makes it overflow.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00 111





Dynamic Hashing

❑ Example

Solution: Here Bucket depth is less than global depth. So, only the bucket split is executed. Elements (19, 1, 5, 7) are re-hashed according to the global depth, which is still 3.

Data	Binary Form
28	11100
4	00100
19	10011
1	00001
22	10110
16	10000
12	01100
0	00000
5	00101
7	00111

Buckets

